Project Integration and End-to-End Practical

Learning outcomes

• Connect requirements, design, implementation, testing, project planning, Git, and Jira in one workflow.

• Trace a user requirement from a story to design choices, code changes, test evidence, and a release decision.

• Use a small end-to-end project to demonstrate engineering practices rather than treating each artefact as separate paperwork.

• Identify dependencies, risks, defects, and completion evidence.

• Prepare a concise project handover and demonstration.

Practical project: College Event Management System (CEMS). Students can browse published events and register when seats are available. Organizers can publish events and manage capacity. This chapter uses a small registration feature to connect the full software engineering lifecycle.

1. What Does End-to-End Integration Mean?

An end-to-end project connects the decisions and artefacts created at each stage of software development. A requirement should influence the design; the design should guide implementation; tests should verify the required behaviour; project tracking should show the work and risks; and release documentation should explain what was delivered.

If these elements are disconnected, a team may finish coding without proving that the correct requirement was implemented, or a test may pass without anyone knowing which user need it protects.

Stage	Key question	Example evidence
Requirements	What problem must be solved?	Approved story and acceptance criteria
Design	How will the solution meet the need?	Architecture sketch, API contract, data model
Planning	Who will do what, and in what order?	Backlog, estimates, dependencies, Sprint Goal
Implementation	What changed in the product?	Code, review, commit history
Testing	How do we know it behaves correctly?	Test cases, results, BDD scenarios, defect records
Tracking	What is the state of the work?	Jira issue status, blockers, linked work
Release / handover	What is ready, and what remains?	Release notes, known issues, user instructions

2. Define the Requirement

Business need: Students need a reliable way to reserve a place at a college event without creating duplicate registrations or exceeding capacity.

User story: As a student, I want to register for an event with available seats so that I can reserve my place.

Acceptance criteria:

• A signed-in student can register for a published event when a seat is available.

• Successful registration creates a confirmation reference.

• A student cannot create a duplicate confirmed registration for the same event.

• When an event is full, the system rejects a new registration with a clear message.

• The registration remains available when the student returns to view their registrations.

These criteria describe observable outcomes. The team should clarify unresolved policy questions—such as cancellation rules, waitlists, or whether registration closes at a particular time—before assuming how the system should behave.

3. Design the Feature

A simple architecture

A browser-based CEMS feature might use a user interface, an application API, and a database. The browser collects user intent and displays outcomes; the API validates the request and enforces business rules; the database stores events and registrations.

```
Browser / UI → Registration API → Business rules and capacity check → Database
The API returns a success response with a confirmation reference, or a clear rejection when
registration is not allowed.
```

> Important design principle: capacity must be enforced on the server, not only by hiding or disabling a button in the browser. Two students might try to reserve the last seat at nearly the same time, so the design must prevent both requests from claiming the same final seat.

Example data concepts

Entity	Example fields	Purpose
Event	eventId, title, status, capacity	Defines the event and its seat limit
Registration	registrationId, eventId, studentId, status, createdAt	Records a student's registration
Student	studentId, name, accountStatus	Identifies the signed-in student

A robust design may require a database constraint or transaction strategy to protect against duplicate registrations and concurrent capacity changes. The exact implementation depends on the chosen database and technology stack.

4. Plan the Work

Break the feature into work items small enough to understand and track. The hierarchy below is illustrative; available parent-child relationships depend on the team's Jira configuration.

Type	Work item	Completion evidence
Epic	Student Event Registration	Related work is grouped and traceable
Story	Register for an available event	Acceptance criteria are satisfied
Task	Define registration API contract	Request, response, and error behavior documented
Task	Implement server-side capacity check	Code reviewed and relevant tests pass
Task	Build confirmation UI	Success and failure states are clear
Task	Add automated registration tests	Normal, full-capacity, and duplicate cases covered
Bug	Full event accepts another registration	Defect reproduced, fixed, and regression-tested

> **Example Sprint Goal:** A signed-in student can complete one reliable registration when seats are available.

5. Manage the Work in Jira

Use the board to make progress and blockers visible. A simple workflow is To Do → In Progress → In Review/Test → Done.

• Each issue has a clear summary, description, and acceptance criteria where relevant.

• Dependencies are recorded so that work is not started on false assumptions.

• A blocker is raised early and assigned an owner for resolution.

• A Bug records expected behavior, actual behavior, reproduction steps, and evidence.

• Done means the agreed Definition of Done is satisfied—not merely that coding has stopped.

Jira tracks work; it does not itself prove quality. Link or record the test evidence and code review that support completion.

6. Use Git for Version Control

Git records changes to project files over time. It helps team members collaborate, review changes, identify when a defect appeared, and return to a known version if needed.

A simple workflow

```
1. Update the local project.
2. Create a branch for the feature or bug.
3. Make a focused change.
4. Run relevant tests.
5. Commit with a meaningful message.
6. Push the branch and open a pull request or merge request.
7. Review, resolve feedback, and merge according to team policy.
```

> Example commit message: **Prevent registration when event capacity is full**. Avoid messages such as “changes”, “final final”, or “stuff”. Never commit passwords, API keys, or private student information.

A commit records a change; it does not guarantee that the change is correct. Review and testing remain necessary.

7. Create a Test Set from Acceptance Criteria

Requirement / criterion	Test scenario	Expected result
Student can register when seats remain	Signed-in student registers for a published event with one seat available	Registration is confirmed and a reference appears
Duplicate registration is prevented	Student registers, then repeats the same action	No second confirmed registration is created
Full event rejects registration	Attempt registration after capacity is reached	Request is rejected with a clear message
Registration can be retrieved	Student opens their registrations after signing in again	Saved registration is shown accurately
Concurrent last-seat attempts are safe	Two requests compete for one remaining seat	At most one new confirmed registration is created

Testing should cover normal behaviour, invalid inputs, boundaries, failure handling, and relevant security or concurrency risks. Test data should be controlled and should not contain unnecessary real student information.

8. Behaviour Scenario Example

```
Feature: Student event registration
Scenario: Registration succeeds when a seat is available
Given the student is signed in
And the event is published and has an available seat
When the student registers for the event
Then the system confirms the registration
And displays a unique registration reference

Scenario: Registration is rejected when the event is full
Given the event has no available seats
When the student attempts to register
Then the registration is not confirmed
And a clear full-event message is displayed
```

These are behaviour examples in a readable format. They become executable automated checks only when the team creates compatible step definitions, test setup, and automation code.

9. Defect Handling and Regression

Suppose a test reveals that a full event still accepts a registration. The team should record a Bug, include reproducible steps and actual versus expected results, assess its impact, fix the underlying cause, and repeat the failing test.

1. Reproduce the defect consistently and capture useful evidence.

2. Create or update the Jira Bug and link it to the related Story.

3. Investigate the cause; do not simply hide the registration button if the server still accepts invalid requests.

4. Implement a fix and request review.

5. Run the failing test again, then run related regression tests.

6. Update issue status and release notes if the fix affects a released version.

> Regression testing checks that a change has not broken previously working behaviour. A defect is not fully addressed until the fix is verified in the relevant context.

10. Definition of Done and Release Readiness

• Acceptance criteria have been checked and satisfied.

• Code review is complete according to team practice.

• Relevant automated and manual tests have passed.

• The feature is integrated into the agreed branch or build.

• No unresolved critical defect blocks release.

• Documentation is updated where necessary.

• The team can demonstrate the feature and explain known limitations.

> A release decision should consider risk, unresolved defects, security, operational readiness, and the intended audience. “Works on my machine” is not sufficient evidence for release readiness.

11. End-to-End Team Activity

In groups, complete a small project packet for the CEMS registration feature. If software tools are unavailable, produce the same artefacts using documents and a paper board.

1. Write one user story and at least four acceptance criteria.

2. Draw a simple architecture diagram and identify the main data entities.

3. Create an Epic, Story, two Tasks, and one Bug in Jira or a Jira-style table.

4. Create a Git branch plan and write two meaningful example commit messages.

5. Write at least four test cases, including a boundary or concurrency case.

6. Write two Given–When–Then scenarios.

7. Define a Definition of Done and identify three project risks.

8. Prepare a three-minute demonstration plan and a short handover note.

> **Submission checklist:** requirement, design, plan/backlog, version-control plan, test evidence, defect record, Definition of Done, and demonstration/handover.

12. Review Questions with Suggested Answers

1. What does end-to-end integration mean?

Suggested answer: Connecting requirements, design, implementation, tests, planning, tracking, and release evidence.

2. Why should capacity be enforced on the server?

Suggested answer: Client-side checks can be bypassed or become stale; the server must enforce business rules safely.

3. How does Git help a team?

Suggested answer: It records versions, supports collaboration and review, and helps identify changes.

4. What makes a useful Bug report?

Suggested answer: Expected and actual results, reproduction steps, relevant context, and evidence.

5. What is regression testing?

Suggested answer: Checking that a change has not broken previously working behaviour.

6. Does a Jira status of Done prove a feature is correct?

Suggested answer: No. Completion should be supported by the Definition of Done and appropriate review and test evidence.

7. Why link requirements and tests?

Suggested answer: It provides traceability and helps show which user needs have been verified.

8. What is a Definition of Done?

Suggested answer: A shared set of quality conditions that work must meet to count as complete.

9. What is a release note?

Suggested answer: A summary of delivered changes and relevant known issues.

10. Why test concurrent last-seat registration?

Suggested answer: To check that simultaneous requests cannot exceed event capacity.

13. Glossary

Term	Meaning
End-to-end traceability	Links from requirements through design, implementation, tests, and release
API contract	Expected request, response, and error behavior at a software interface
Version control	Recording and managing changes to files over time
Commit	A recorded set of changes in a version-control repository
Pull / merge request	A request to review and integrate a proposed change
Regression test	A check that a change has not damaged existing behaviour
Defect	A flaw causing actual behaviour to differ from expected behaviour
Release readiness	Evidence that a version is suitable for its intended release
Definition of Done	Shared quality conditions required for work to be considered complete

14. Homework

1. Choose a Library Management System or Hospital Appointment System and trace one requirement through design, implementation tasks, tests, and a release note.

2. Write five test cases and two behaviour scenarios for the chosen requirement.

3. Prepare a Jira-style issue hierarchy and explain the dependencies.

4. Write a Definition of Done appropriate for a student software project.

5. Reflect on one risk that could remain hidden if requirements, code, tests, and project tracking were maintained separately.

Next chapter: Day 29 — Project Review, Quality Audit, Risk Review, and Final Demonstration Preparation.